



RAG Pipeline Fundamentals

From Documents to Grounded Answers

A comprehensive guide to Retrieval-Augmented Generation — how to connect language models to your organization's private knowledge so they answer with evidence, not guesswork.

What We'll Cover Today

01

RAG Foundations

Why LLMs alone fall short and how retrieval changes the equation

02

Document Ingestion & Chunking

Loading, extracting, splitting and tagging your source material

03

Embeddings & Retrieval

Turning text into vectors and finding the most relevant passages

04

End-to-End Pipeline

Indexing vs. query time, prompt anatomy and context assembly

05

Quality & Limitations

Grounding, failure modes, production considerations and when to use RAG

RAG FOUNDATIONS

The Problem: LLMs Don't Know What They Don't Know

Large language models are trained on massive public datasets — but your organization's product manuals, returns policies, and support FAQs were never part of that training. The result: confidently stated answers that are outdated, invented, or simply wrong for your specific context.



Three Core Gaps in LLM-Only Applications

Private Information

Your internal documents, product specs, HR policies, and customer data were never included in the model's training corpus. The model simply has no access to them.

Changing Information

LLMs have a training cutoff. A returns policy updated last quarter, a new product line, or a revised pricing schedule are all invisible to the frozen model weights.

Missing Information

Even for topics covered in training, the model may fill gaps with plausible-sounding fabrications — a behavior known as hallucination — because it has no mechanism to say "I don't have a source for this."



LLMs do not flag uncertainty by default. Without a retrieval system, there is no guarantee that an answer is grounded in any real document.

Defining Retrieval-Augmented Generation

The One-Sentence Definition

RAG is an architecture that retrieves relevant passages from a document collection at query time and supplies them as context to a language model before it generates a response — grounding the answer in real, citable source material.

Breaking Down the Acronym

Retrieval — Search a document collection for passages relevant to a specific question.

Augmentation — Insert those retrieved passages into the prompt as evidence the model can read.

Generation — The LLM reads both the evidence and the question, then produces a grounded, coherent answer.

RAG Does Not Retrain the Model

A common misconception is that RAG "teaches" the LLM new facts by updating its weights. **This is not what happens.**

RAG supplies context at inference time — it places retrieved text inside the prompt window so the model can read and cite it *for that specific request*. The model's parameters remain completely unchanged.

Think of it like handing a knowledgeable colleague the relevant pages from a manual before asking a question. Their underlying expertise doesn't change — but their answer now references real material.



Fine-tuning modifies model weights. RAG does not. They solve different problems and can be combined.

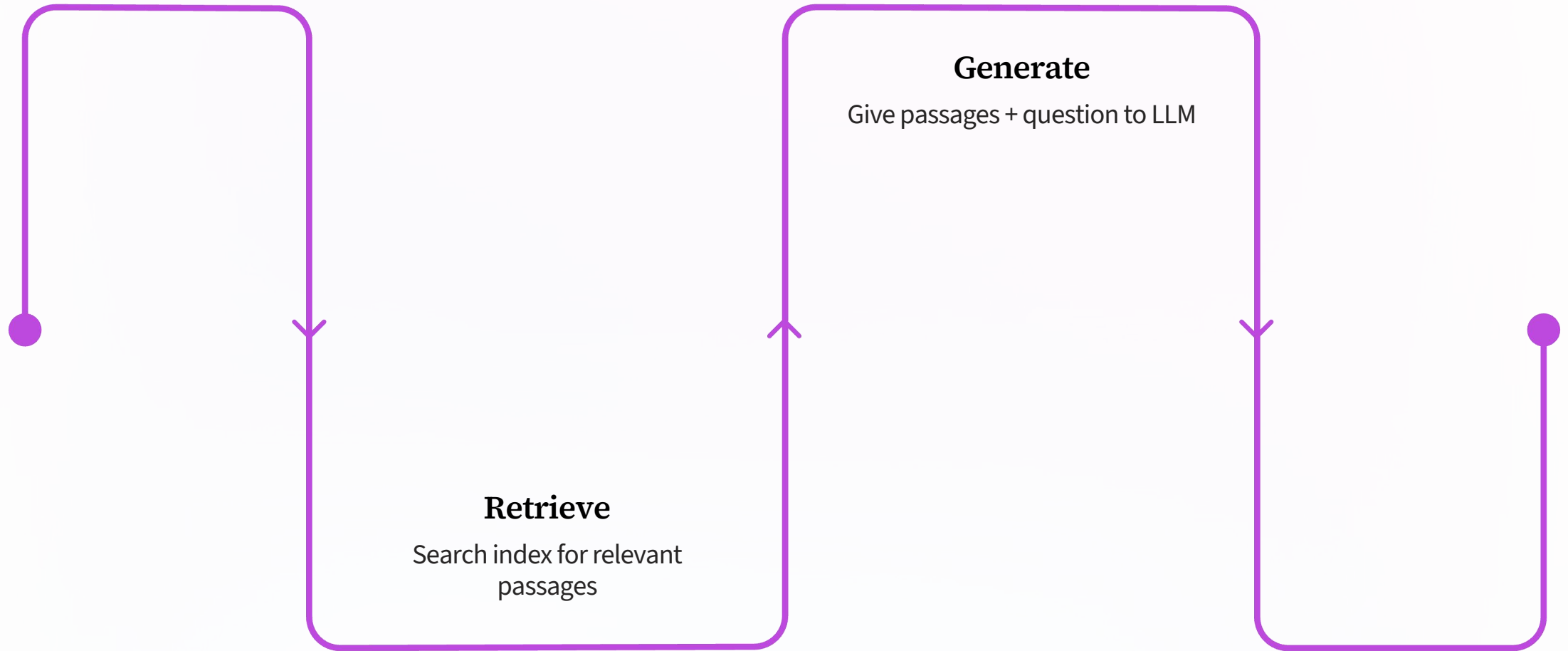
Fine-Tuning

Updates model weights.
Teaches style, format, or domain behavior.
Requires labeled data and retraining compute.

RAG

Supplies context at runtime. Adds private or current knowledge.
Requires a retrieval index and embedding model.

The Basic Two-Step RAG Flow



Every RAG pipeline, regardless of complexity, executes this fundamental sequence. Retrieval brings evidence to the model; generation turns that evidence into a readable, cited answer.

RAG FOUNDATIONS

Our Running Example: An Electronics Retailer

Throughout this course we'll use a mid-size electronics retailer that has three bodies of internal knowledge: **product manuals** for hundreds of devices, a **returns and warranty policy** document updated each quarter, and a growing library of **customer support FAQs**. None of this content appears in any public LLM training set — making it a realistic RAG scenario.



Where RAG Adds Real Value

Policy Assistance

HR, legal, or returns-policy chatbots that must cite the exact clause that governs a decision — and flag when policy has changed.

Enterprise Knowledge Search

Internal tools that let employees query scattered institutional knowledge — wikis, runbooks, meeting notes — in natural language.

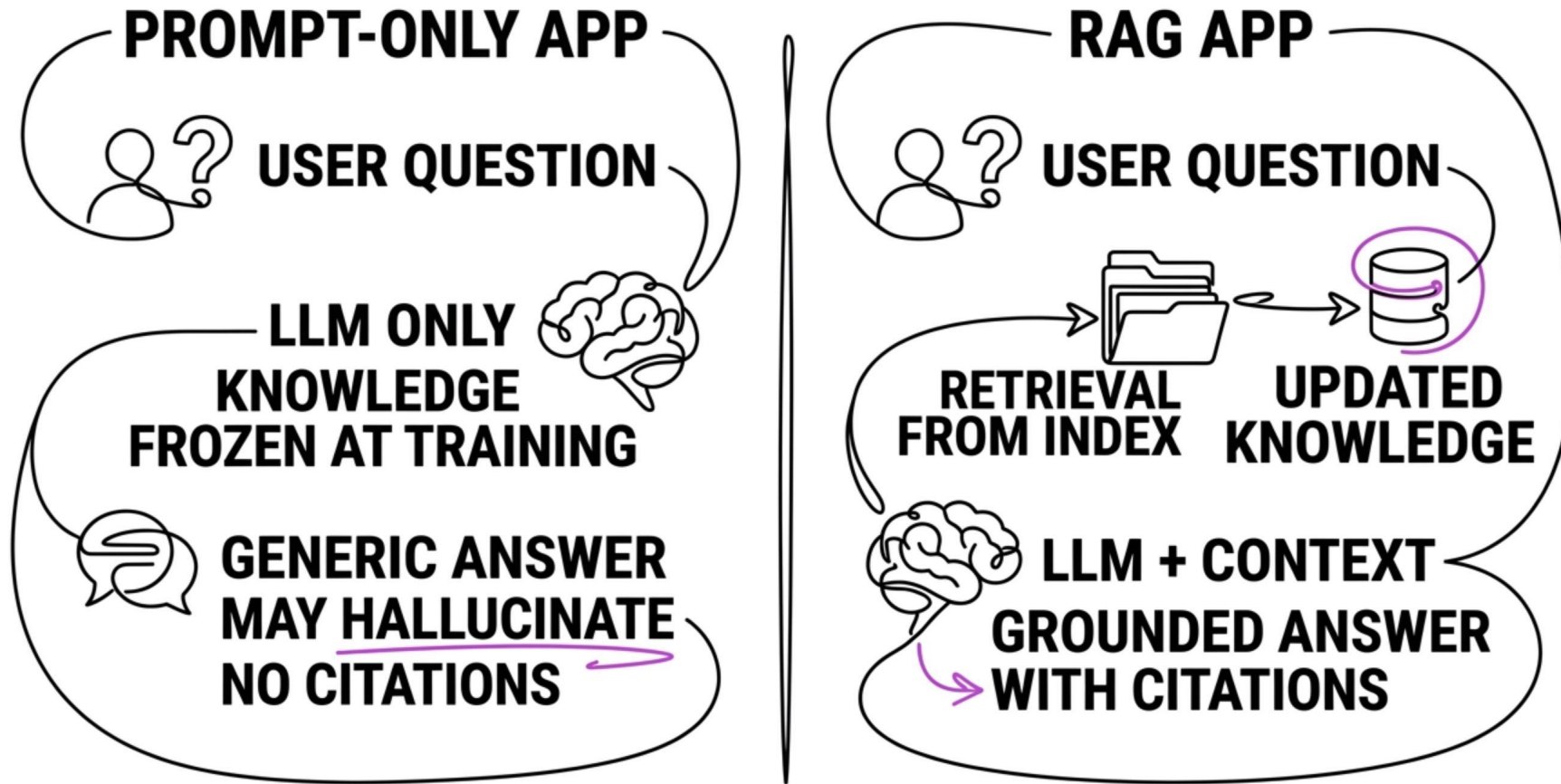
Product Support

Support agents or self-service bots that surface the right paragraph from a product manual rather than paraphrasing from memory.

Document Q&A

Legal, research, or compliance teams that need precise, traceable answers from large document sets without reading everything manually.

Prompt-Only vs. RAG Applications



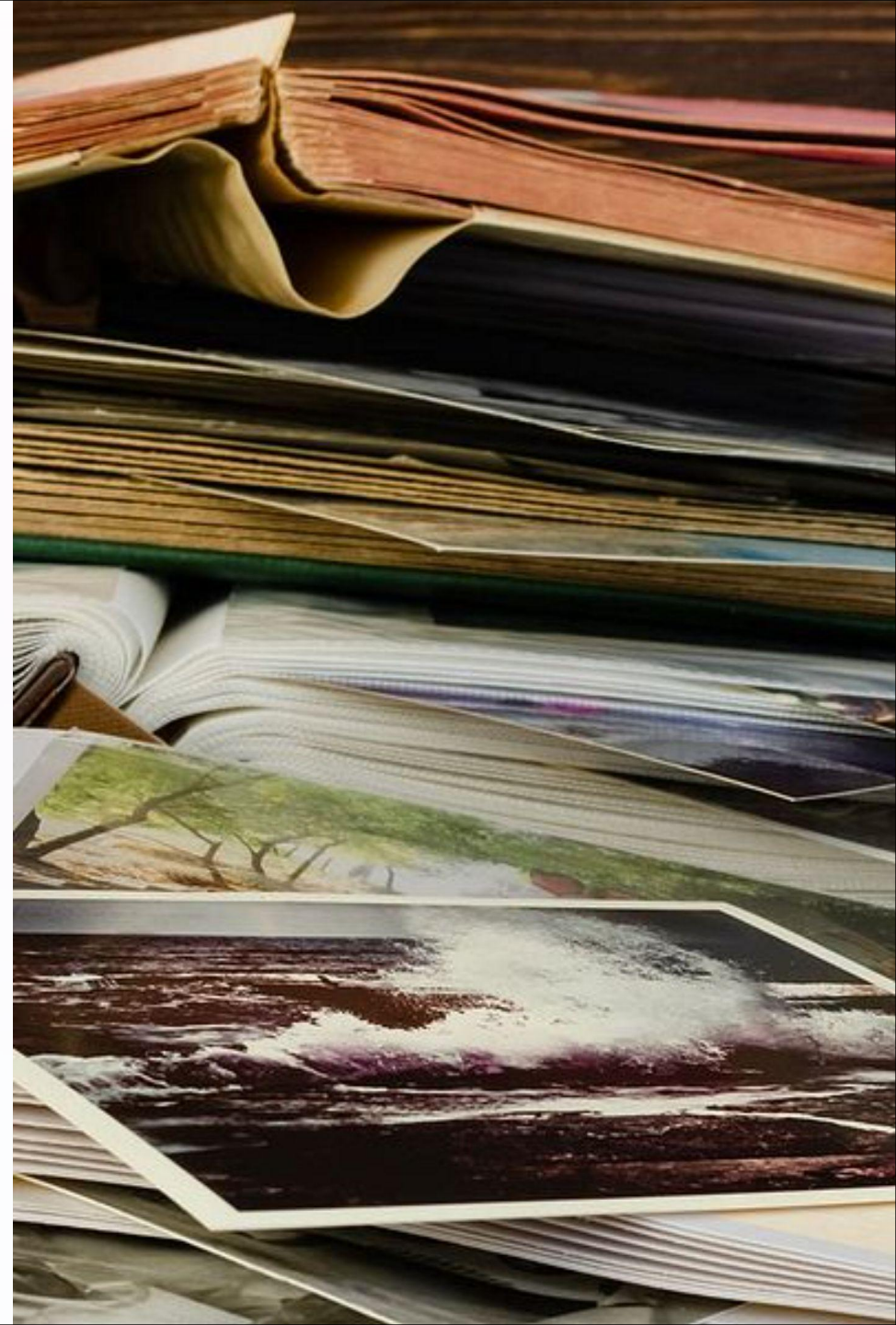
The architectural difference is small but the reliability difference is significant — especially for domain-specific, private, or time-sensitive information.

DOCUMENT INGESTION

Stage 1: Document Ingestion

Before retrieval can happen, your source documents must be loaded into the pipeline.

Ingestion is the process of reading raw files — PDFs, plain-text files, HTML pages, Word documents — and converting their contents into structured Python objects that downstream components can process. This is the entry point of every RAG system.



The LangChain Document Object

What Is a Document?

In LangChain, a **Document** is a lightweight data container with exactly two fields: **page_content** (a string holding the extracted text) and **metadata** (a dictionary of descriptive key-value pairs). Every loader, splitter, and retriever in LangChain passes these objects through the pipeline.

Illustrative Document Object

```
Document(  
  page_content=(  
    "Return window: 30 days from "  
    "purchase date. Item must be "  
    "in original packaging."  
  ),  
  metadata={  
    "source": "returns_policy_v4.pdf",  
    "page": 3,  
    "section": "Standard Returns",  
    "doc_version": "2024-Q2"  
  }  
)
```

Metadata: The Document's Identity Card

Metadata travels with every chunk through the entire pipeline. It enables filtering, citation, and debugging. Common metadata fields and their purposes:

source

File name or URL. Lets the system tell the user *which document* an answer came from — e.g., `returns_policy_v4.pdf`.

page

Page number within the source file. Supports precise citation and lets a user navigate directly to the original passage.

section

Logical section title (e.g., `Warranty Claims`). Useful for retrieval filters — e.g., only search FAQ sections for tier-1 support queries.

doc_version

Version or date stamp. Critical for policies that change quarterly — prevents an old returns window from contaminating a current answer.

Text Extraction Limitations

What Loaders Do Well

Digitally created PDFs and plain-text files yield clean, correctly ordered text. Most LangChain loaders handle these reliably out of the box.

Where Extraction Breaks Down

→ Scanned PDFs

Image-based pages contain no machine-readable text. OCR is required and introduces recognition errors that cascade downstream.

→ Tables and Figures

Column alignment and row relationships are lost when extracted as linear text. A pricing table becomes a garbled sequence of numbers.

→ Reading Order

Multi-column layouts, sidebars, and footnotes can be extracted out of order, creating passages that make no sense as prose.



Why We Chunk Documents

The Core Problem

A 50-page product manual cannot fit in an LLM's context window. Even if it could, embedding an entire document produces a single vector that averages all its topics — making precise retrieval impossible. You need small, focused passages.

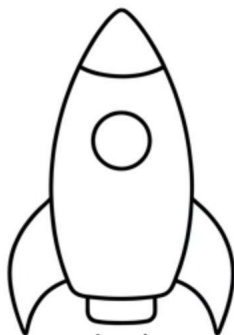
What Chunking Achieves

- **Fits context windows** — each chunk is small enough to pass to an LLM alongside other chunks and the question.
- **Enables precise retrieval** — a focused passage about "30-day return window" scores higher for that query than a whole policy document.
- **Creates granular citations** — you can point the user to page 3, section 2, rather than "see the manual."

Chunking is the process of splitting a loaded Document into a series of smaller Document objects, each carrying the original metadata plus its position within the parent document.

Chunk Size and the Precision–Context Tradeoff

**Precise
Retrieval,
but risks mid-
concept splits.**



Small Chunks

**Balanced
performance,
good for most
prose.**



Medium Chunks

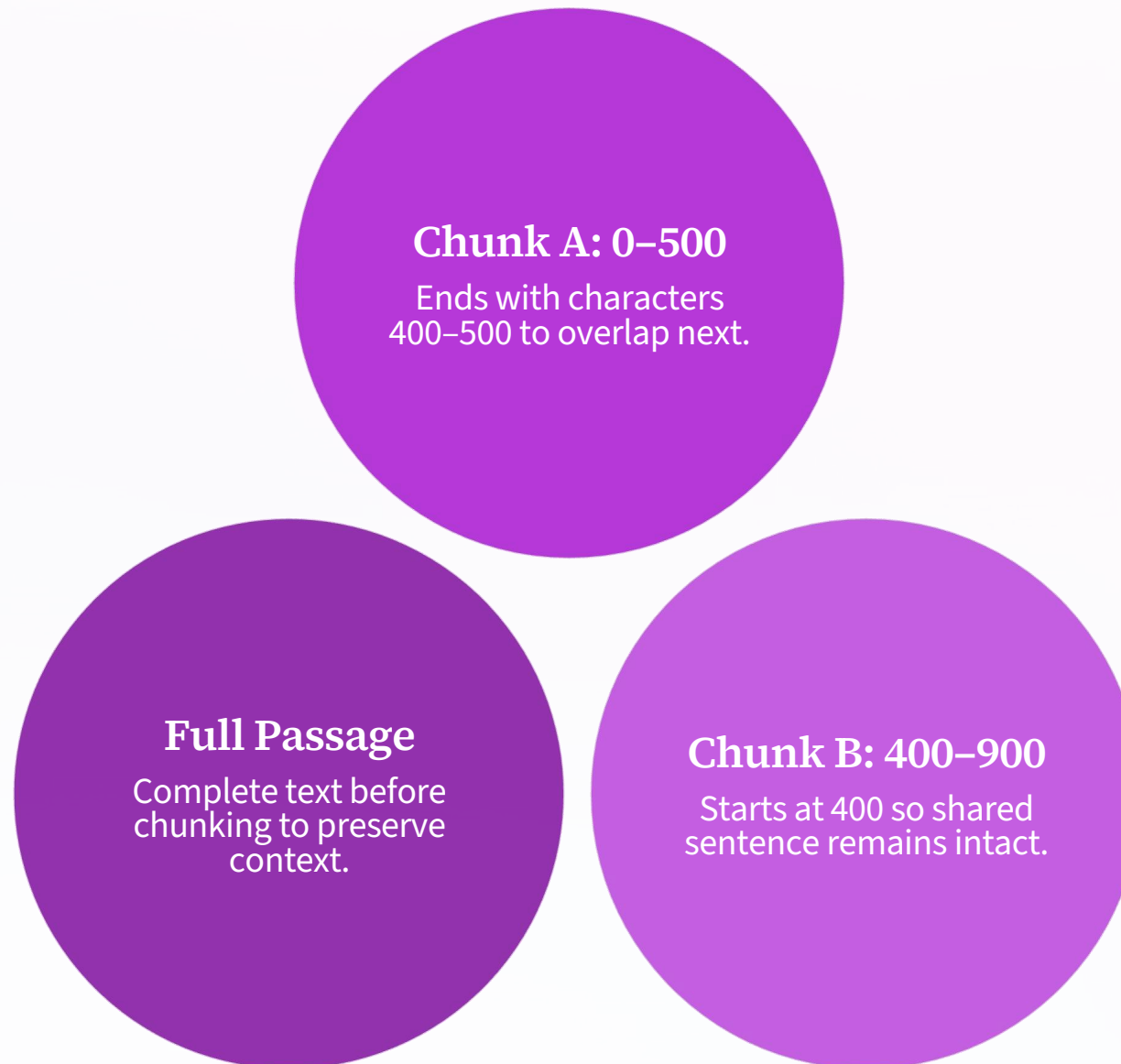
**Rich Context,
but risks
irrelevance and
token overflow.**



Large Chunks

Chunk size is measured in **characters** or **tokens** depending on the splitter. There is no universal optimal size — the right value depends on document structure, typical question specificity, and your LLM's context window. Split dense topics to avoid shrinking text rather than choosing an extreme chunk size.

Chunk Overlap: Preserving Information at Boundaries



RecursiveCharacterTextSplitter: How It Decides Where to Cut

The Separator Hierarchy

LangChain's `RecursiveCharacterTextSplitter` tries a prioritized list of separators in order, falling back to finer splits only when necessary:

01

Double newline `\n\n`

Split at paragraph boundaries first — the most semantically clean cut.

02

Single newline `\n`

Used when a paragraph is still too large for the target chunk size.

03

Space

Word boundary splits when line-level cuts still produce oversized chunks.

04

Character

Last resort — splits mid-word. Rarely ideal but guarantees the size limit is respected.

What It Does Not Do

This splitter operates purely on character patterns. It does **not** understand document meaning, topic boundaries, or sentence semantics. It cannot recognize that a sentence about warranty terms is more related to the next sentence than to a paragraph four lines below.

 Semantic chunking strategies — which use embeddings to detect topic shifts — exist but add complexity. `RecursiveCharacterTextSplitter` remains the practical default for most projects.

Metadata Must Travel With Every Chunk

When a splitter divides a Document, it should propagate the parent's metadata to every resulting chunk. A chunk that loses its `source` or `doc_version` field cannot be correctly cited and cannot be filtered during retrieval.

Parent Document

`source: "manual_ProTV55.pdf"`

`page: 12`

`section: "Connectivity"`

Chunk A

`source: "manual_ProTV55.pdf"`

`page: 12`

`section: "Connectivity"`

`chunk_index: 0`

Chunk B

`source: "manual_ProTV55.pdf"`

`page: 12`

`section: "Connectivity"`

`chunk_index: 1`

Every splitter in LangChain copies metadata by default, but custom ingestion pipelines must explicitly carry these fields through each transformation step.



EMBEDDINGS & RETRIEVAL

Stage 2: Embeddings

Once documents are chunked, each chunk is converted into a **numerical vector** — a list of floating-point numbers — by an embedding model. These vectors capture the meaning of the text in a form that allows mathematical comparison. Similar meanings produce vectors that are close together; unrelated meanings produce vectors that are far apart.

What Is an Embedding?

A Numerical Representation of Meaning

An embedding model maps a string of text to a fixed-length vector — typically hundreds to thousands of dimensions. Every position in the vector captures some aspect of the text's semantics, though individual dimensions are not human-interpretable.

Embedding models used in RAG are **separate from the generative LLM**. Common choices include OpenAI's `text-embedding-3-small`, Cohere Embed, or open-source models like `all-MiniLM-L6-v2`. They are optimized for comparison tasks, not text generation.

 Token representations inside a generative LLM — sometimes also called "embeddings" — serve a different purpose and are not used for retrieval.

Illustrative Vector (Simplified)

Text:

"30-day return window for
unopened items."

Vector (1536-d, shown as 8):

[0.21, -0.44, 0.87, 0.03,
-0.12, 0.55, -0.38, 0.76,
...]

Real vectors have hundreds or thousands of dimensions and are not human-readable.

Semantic Similarity vs. Keyword Matching

Keyword Search

Matches documents containing the exact terms in the query. A search for *"return policy"* would miss a chunk that says *"items may be sent back within 30 days"* — even though the meaning is identical. Precision depends on word choice.

Semantic Search (Embeddings)

Compares vector representations of meaning. *"Can I send this back?"* and *"What is the return window?"* produce similar vectors and retrieve the same policy chunk — even though they share no keywords. Meaning drives retrieval, not exact wording.

This is the core advantage of embedding-based retrieval for user-facing Q&A: customers rarely phrase questions the way documents are written.

A Schematic Embedding Map



The Vector Store

Definition

A **vector store** is a specialized database that stores embedding vectors alongside the original text and metadata of each chunk. Its primary operation is **similarity search**: given a query vector, it returns the k stored vectors with the highest similarity score — along with their associated text and metadata.

What Lives in the Vector Store

Vector

The numerical embedding of the chunk text.

page_content

The original chunk text, returned alongside the vector.

metadata

Source, page, section, version — used for filtering and citation.

Popular vector stores include **FAISS** (in-memory, open-source), **Chroma** (local or server), **Pinecone** (managed cloud), and **pgvector** (PostgreSQL extension). LangChain provides a consistent interface across all of them.

Query and Document Embeddings Must Be Compatible

Similarity search only produces meaningful results when the **query embedding** and the **document embeddings** were produced by the **same model** — or at minimum, models trained to produce compatible vector spaces.

✓ Compatible

All chunks embedded with `text-embedding-3-small`. All queries embedded with `text-embedding-3-small`. Similarity scores are meaningful.

✗ Incompatible

Chunks embedded with Model A. Queries embedded with Model B. Different vector spaces — similarity scores are numerically meaningless. Retrieval will fail silently.

⚠ Switching embedding models requires re-embedding and re-indexing the entire document collection. Plan your embedding model selection carefully before populating a production index.

Similarity Search, Scores, and top-k Retrieval

How Similarity Is Measured

The most common metric is **cosine similarity**: the cosine of the angle between two vectors, ranging from -1 (opposite) to 1 (identical direction). A score of 0.89 between a query and a chunk means they are strongly aligned in meaning. It does **not** mean there is an 89% probability that the chunk correctly answers the question.



Similarity scores are not probabilities of correctness. A high score means the chunk is topically related — not that it contains a correct or complete answer.

top-k: Choosing How Many Chunks to Retrieve

The retriever returns the k highest-scoring chunks. The tradeoff is real in both directions:

- **k too small** — you risk missing a chunk that contains essential evidence, especially when an answer spans multiple passages.
- **k too large** — you fill the context window with irrelevant passages, dilute the prompt, and may lead the LLM to synthesize information it shouldn't.

A typical starting range is $k=3-6$. The right value depends on document structure and query complexity.

The Retriever: An Interface, Not Just a Database

What Is a Retriever?

In LangChain, a **retriever** is an abstraction that accepts a query string and returns a list of Documents. It sits above the vector store and can implement any retrieval strategy — similarity search, keyword search, hybrid approaches, or self-querying — behind a uniform interface.

The vector store is the database. The retriever is the search logic that queries it.

Retriever vs. Vector Store

Concern	Vector Store	Retriever
Stores vectors		
Runs similarity search		Delegates
Returns Documents	Sometimes	
Supports custom logic		
Used in LCEL chains	Rarely direct	

Retrieving k Results $\neq$ Guaranteeing an Answer

Case 1: Answer in One Chunk

Query: *"What is the standard return window?"*

Chunk 1 (score 0.91): *"Items may be returned within 30 days of purchase..."*

→ Complete, grounded answer available.

Case 2: Answer Across Several Chunks

Query: *"Can I return a damaged TV I bought online?"*

Chunk 1: online purchase rules. Chunk 3: damage exception clause. Chunk 5: restocking fees.

→ Answer requires synthesis of multiple retrieved chunks.

Case 3: Answer Absent

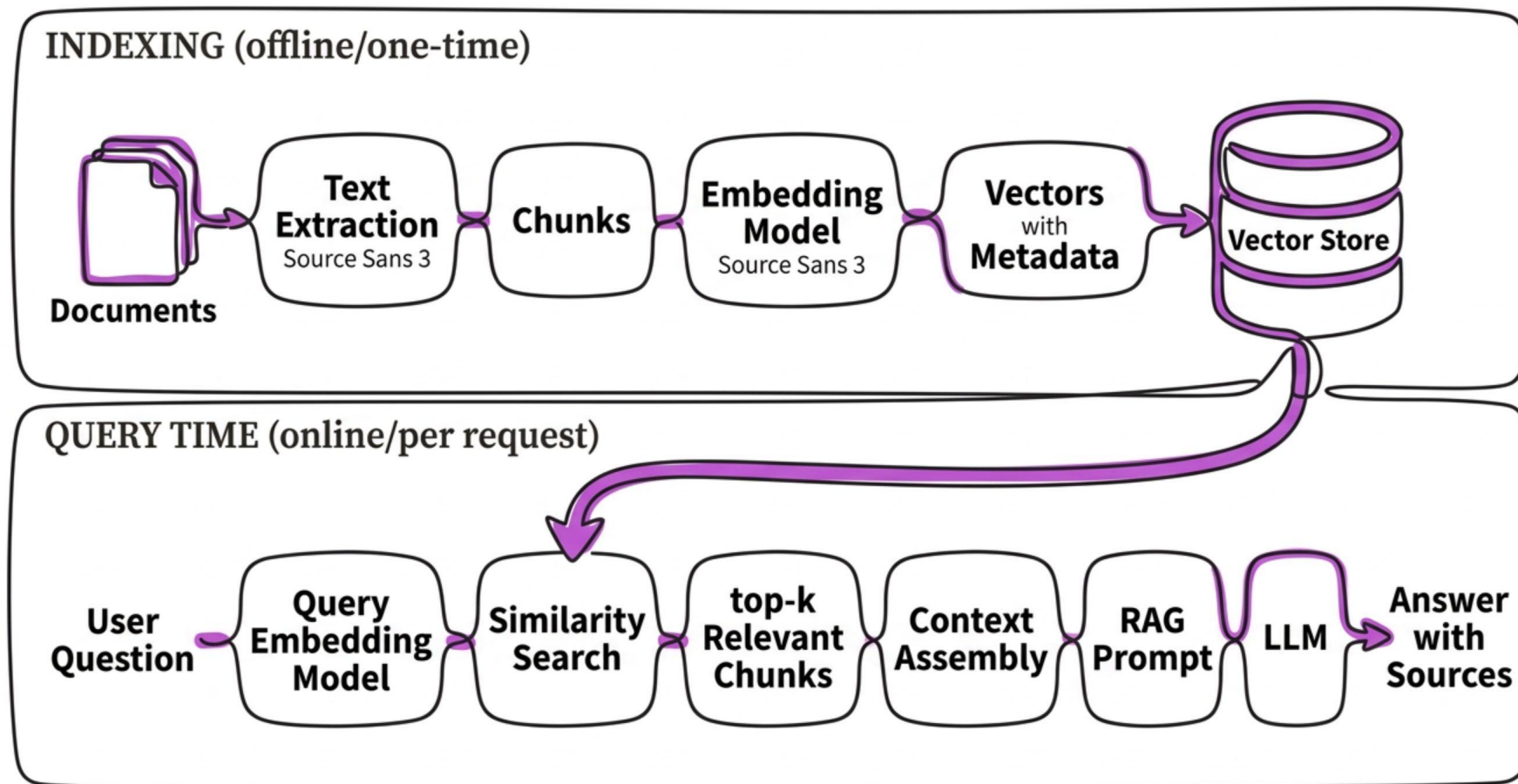
Query: *"Do you offer trade-in credit for old devices?"*

Top chunks all score below 0.6 and discuss return windows, not trade-ins.

→ No retrieved chunk supports an answer. System should say so explicitly.

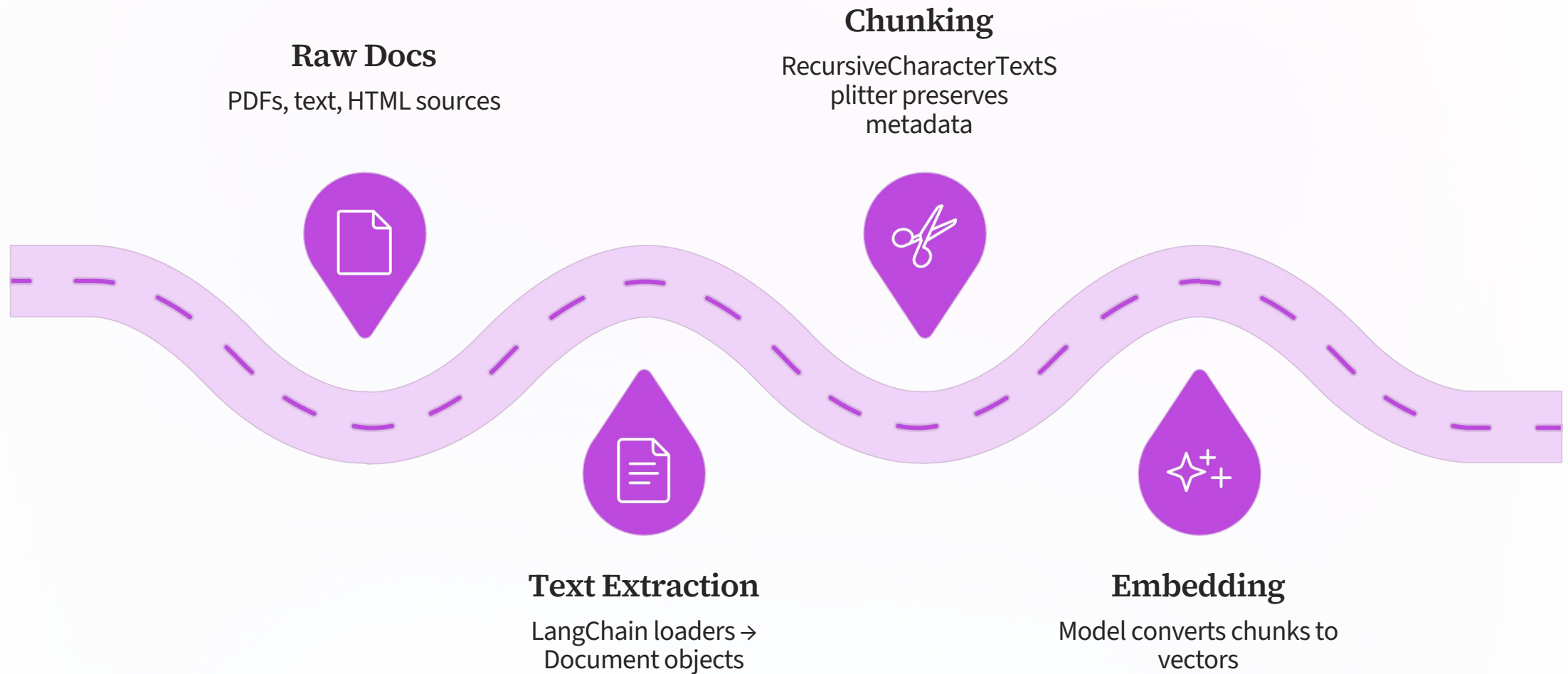
The retriever always returns k results — even when none of them contain a relevant answer. The LLM must be instructed to recognize and report this situation.

Two Distinct Phases: Indexing vs. Query Time



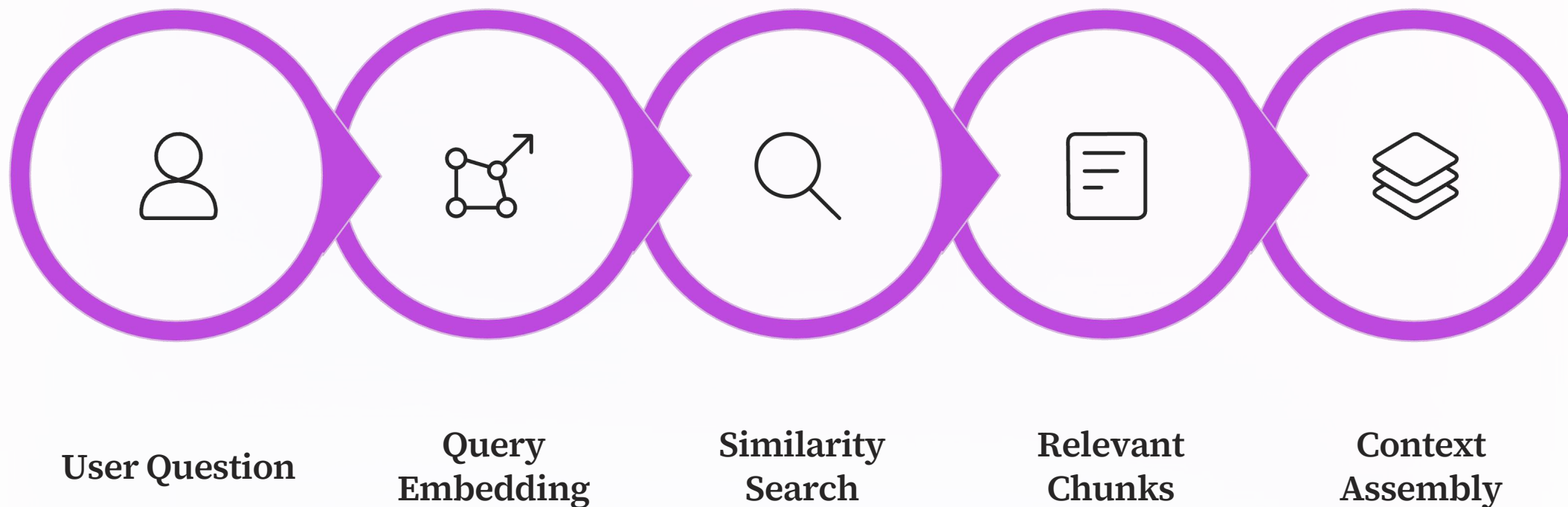
Indexing is performed once (or on document updates) and is computationally expensive. Query time runs on every user request and must be fast. The vector store is the bridge between the two phases.

Indexing: Building the Knowledge Base



This pipeline runs **offline** — before any user asks a question. The output is a populated vector store ready to serve similarity searches. Whenever source documents change, the relevant portions of the index must be refreshed.

Query Time: Answering a User Question



The embedding model appears **twice** in the full system: once during indexing (to embed chunks) and once at query time (to embed the question). The generative LLM is used only at query time.

Context Assembly: What Goes Into the Prompt

The Assembly Problem

Retrieved chunks must be formatted and ordered before being inserted into the prompt. Key considerations: **ordering** (most relevant first or recency-ordered?), **source labels** (each chunk labeled with its source and page for citation), and **token budget** (chunks plus question plus instructions must fit within the LLM's context window).

Token Budget Considerations

- **System instructions** consume tokens before any evidence appears.
- **Each chunk** contributes its character count, converted to tokens (roughly 4 characters per token for English).
- **The question** itself adds tokens.
- **Reserved space** for the model's output must be accounted for.

Exceeding the context window causes truncation — silently dropping evidence the model needed. Monitor total prompt length in production.

Anatomy of a RAG Prompt

1 • Task Instruction

"You are a helpful assistant for [Electronics Retailer] customer support." — Sets role and scope.

2 • Supplied Evidence

Retrieved chunks, each prefixed with a source label: [Source: returns_policy_v4.pdf, p.3] — the model should read these as its only knowledge base for this question.

3 • User Question

"Can I return a 55-inch TV I bought online if the screen is cracked?"

4 • Grounding Instructions

"Answer only from the provided documents. Do not use outside knowledge." — Prevents hallucination by restricting the model's source.

5 • Answer-Not-Found Behavior

"If the documents do not contain enough information to answer, say so clearly and do not speculate."

Grounding: Answering Only From Supplied Context

What Grounding Means

Grounding is the constraint that an LLM's answer should be traceable to specific passages in the supplied context — not to the model's parametric memory or general world knowledge. A grounded answer cites the chunk it drew from. An ungrounded answer may sound correct but cannot be verified.

Grounding is enforced through **prompt instructions** and evaluated through **faithfulness metrics** — it is not a property the model guarantees automatically.

LLM-Only vs. RAG Answer (Illustrative)

✗ LLM-Only (Unsupported)

"Most retailers accept TV returns within 14 to 30 days." — Plausible, but not specific to this retailer and cannot be cited.

✓ RAG (Grounded)

"According to the Returns Policy (p.3), damaged items are not eligible for standard return. A warranty claim may apply — see the Warranty section (p.7)."

Retrieval Relevance $\neq$ Answer Faithfulness

Two distinct quality dimensions are easy to confuse — but they fail independently:

Retrieval Relevance

Did the retriever return chunks that are *topically appropriate* for the question?
Measured against a set of known-relevant documents. A retriever can score well here while the LLM still produces a bad answer.

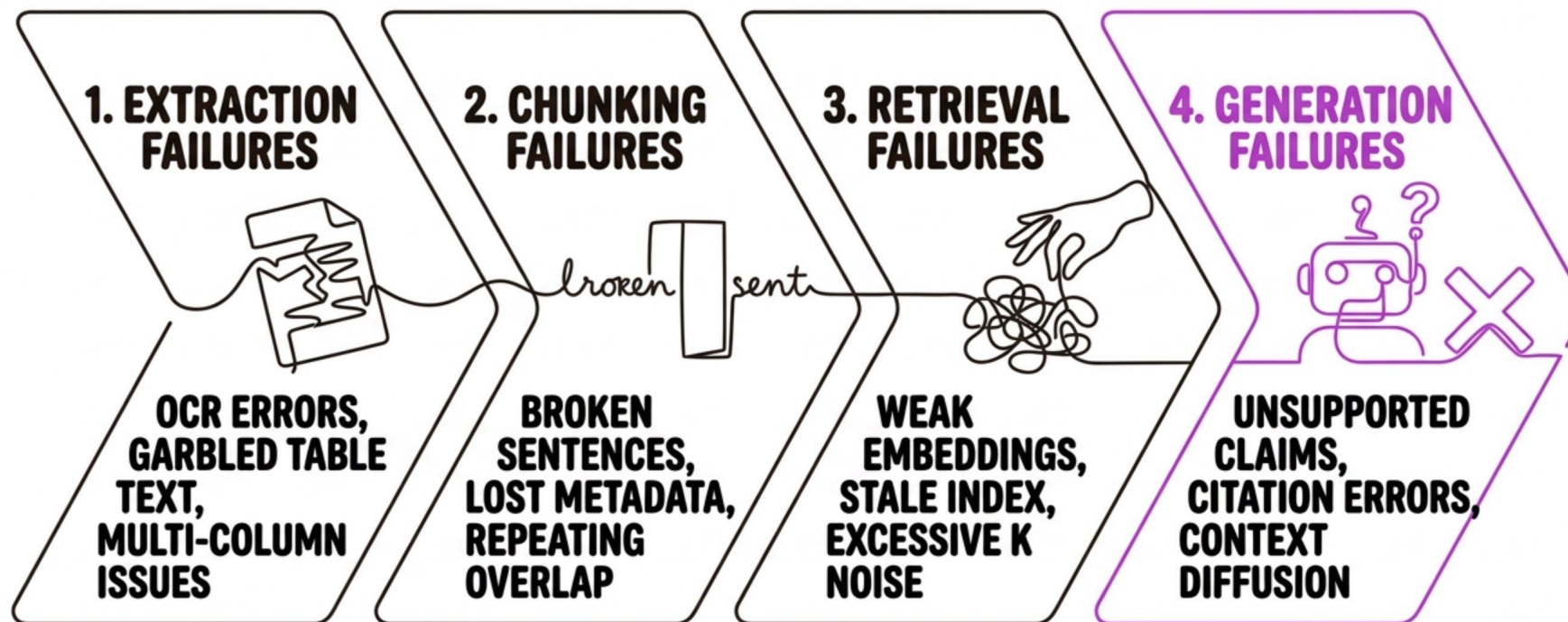
Answer Faithfulness

Does the generated answer stay true to what the retrieved chunks actually say? A faithful answer introduces no content absent from the context. High faithfulness with low retrieval relevance still produces a wrong answer — just a wrong answer that accurately paraphrases irrelevant chunks.

Answer Correctness

Is the answer actually right? Requires both relevant retrieval and faithful generation — and an independent ground truth to evaluate against. This is the hardest quality dimension to measure automatically.

Where RAG Pipelines Fail: A Failure Map



Every stage of the pipeline can degrade answer quality independently. Systematic evaluation at each stage is necessary to diagnose root causes when answers are wrong.

How Chunk Size, Overlap, and k Interact

They Are Coupled — Not Independent

Changing one parameter shifts the optimal values for the others. Small chunks require a larger k to assemble sufficient context. Large chunks with high overlap amplify duplication at larger k . There is no universal optimal combination.

Practical Interaction Summary

Chunk Size	Effect on Retrieval	Implication for k
Small ($\approx 200\text{--}400$ chars)	Precise, focused passages	May need higher k to cover full answer
Medium ($\approx 600\text{--}1000$ chars)	Balanced precision/context	$k = 3\text{--}5$ is a reasonable starting point
Large ($\approx 1500+$ chars)	Rich context, noisy embeddings	Lower k to avoid filling context window



Evaluate with your actual documents and real user queries. Benchmarks from other domains rarely transfer directly.

Citations Must Actually Support Their Claims

The Citation Problem

An LLM can generate a source label ([returns_policy_v4.pdf, p.3]) next to a claim even when the cited chunk does not contain that claim. This is one of the most dangerous failure modes in RAG — it looks trustworthy to users but is unsupported.

A genuine citation means: **the cited passage contains the information stated in the answer**. Anything less is misleading, even if the answer happens to be true.

How to Validate Citations

1

Faithfulness evaluation

Use automated metrics (e.g., RAGAS faithfulness) to check whether each claim in the answer is supported by the retrieved chunks.

2

Human spot-check

Sample answers and manually verify that cited passages contain the claimed information.

3

Structured output

Prompt the LLM to return claim–citation pairs in a structured format, making automated verification easier.



QUALITY & LIMITATIONS

Production Considerations

Moving a RAG prototype to production introduces concerns that go well beyond retrieval accuracy. These must be addressed before real users depend on the system.

Production Considerations: People & Data

Permissions & Privacy

If users have different access levels, the retriever must filter by permission — otherwise a customer could surface internal pricing documents. Never index sensitive data without an access-control layer in front of retrieval results.

Document Updates

Stale chunks cause wrong answers. A quarterly returns-policy update requires re-ingesting and re-embedding the changed document and replacing old chunks in the index. Design your ingestion pipeline to support incremental updates from day one.

Retrieval Filtering

Use metadata filters to scope retrieval — e.g., only search `doc_type: faq` for tier-1 support queries, or only search `doc_version: 2024-Q3` to exclude deprecated policies. This improves precision without requiring a larger k .

Production Considerations: Performance & Trust

Latency & Cost

Every query involves at least two model calls: the embedding model and the generative LLM. Retrieval adds a database round-trip. Monitor and optimize each step separately. Caching common query embeddings and pre-ranking strategies can help at scale.

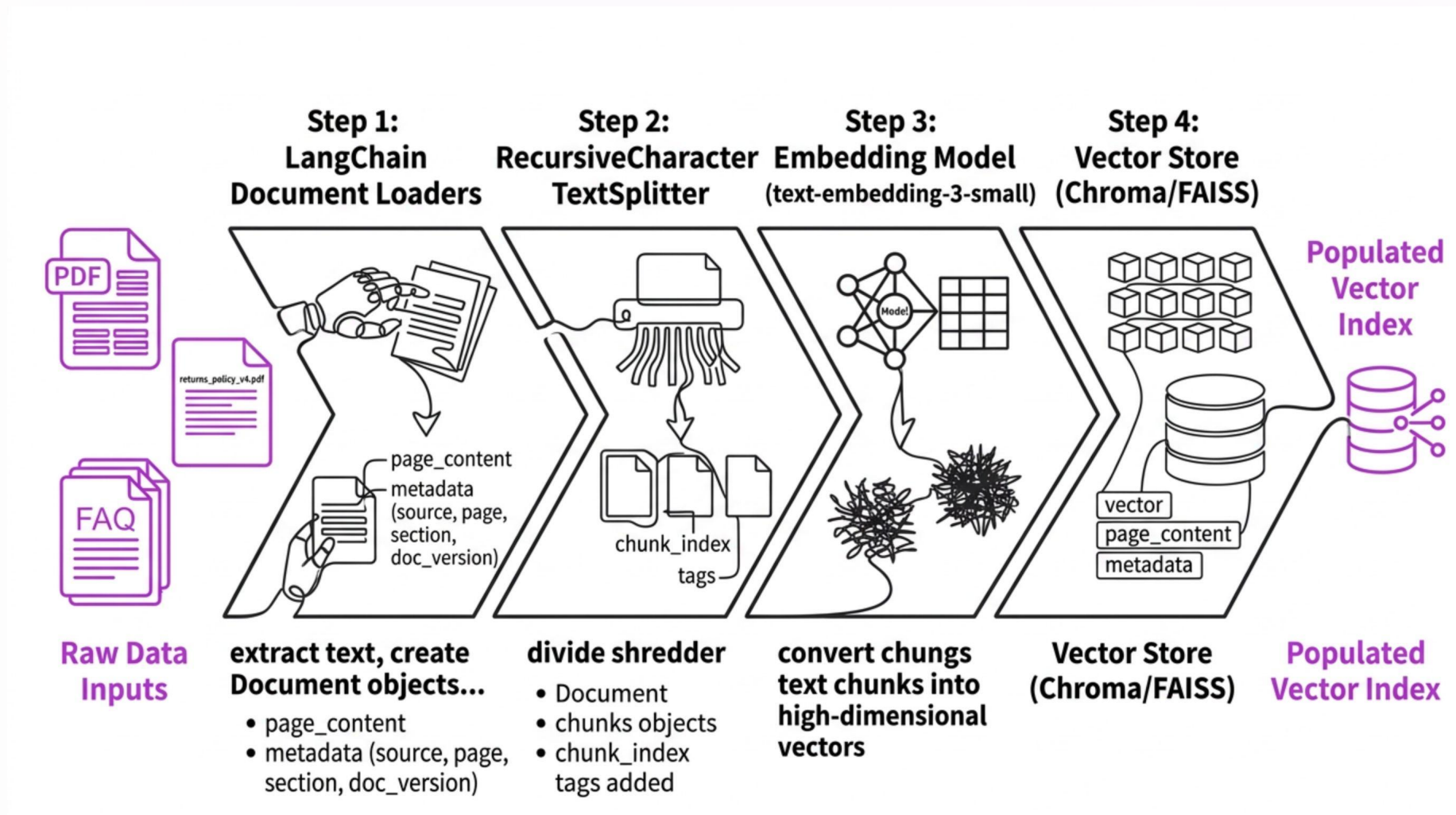
Monitoring & Evaluation

Log query–context–answer triples in production. Use frameworks like **RAGAS** or **LangSmith** to continuously measure retrieval relevance, faithfulness, and answer correctness. Evaluation should be ongoing, not a one-time check at launch.

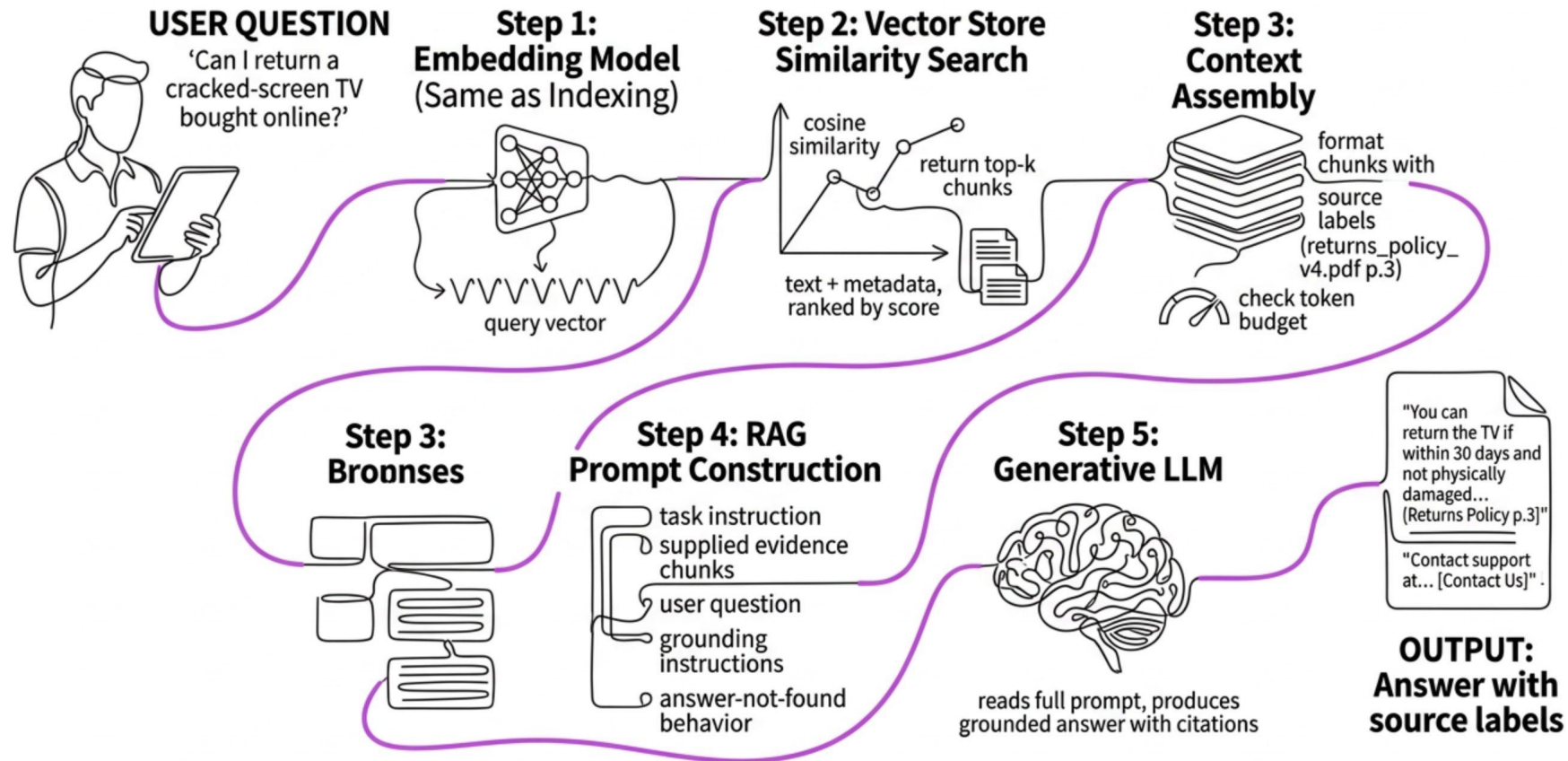
RAG Does Not Eliminate Hallucination

RAG significantly reduces hallucination by anchoring answers to retrieved evidence — but a model can still introduce unsupported claims, especially under adversarial prompting or when grounding instructions are weak. Treat it as a strong mitigation, not a guarantee.

Complete Architecture: Indexing Phase



Complete Architecture: Query Time Phase



Which Components Use Which Model

Component	Model Type	When It Runs
Chunk embedding (indexing)	Embedding model	Offline / document update
Query embedding	Embedding model (same)	Every user request
Similarity search	No model — vector math	Every user request
Answer generation	Generative LLM (e.g., GPT-4o)	Every user request
Context assembly	No model — string formatting	Every user request

Understanding this separation prevents a common confusion: the embedding model and the generative model are distinct systems with different purposes, different providers, and different cost profiles.

WHEN TO USE RAG

When RAG Is the Right Choice

✓ Use RAG When...

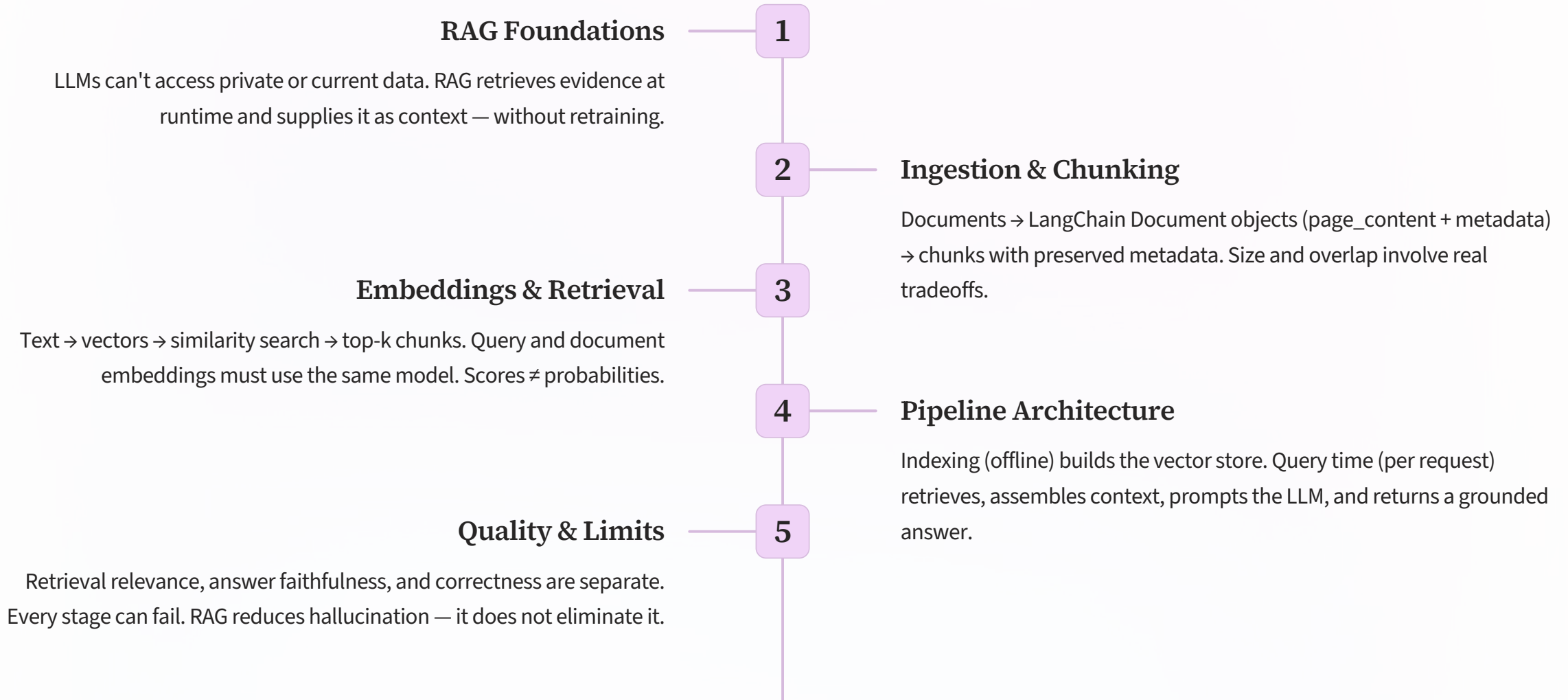
Your knowledge base is private, proprietary, or unavailable in public training data. Content changes frequently. Answers must be traceable to specific documents. Users need citations to trust responses.

⚠ Consider Alternatives When...

The required knowledge is stable general information already well-covered in training data. Response latency requirements are very tight. The document corpus is tiny (a few hundred words). Fine-tuning for style/format is the actual goal.



What We've Covered: The Full Picture



References & Further Reading

Foundational Research

- Lewis et al. (2020) — *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arxiv.org/abs/2005.11401
- Gao et al. (2023) — *Retrieval-Augmented Generation for Large Language Models: A Survey*. arxiv.org/abs/2312.10997

LangChain Documentation

- LangChain Document Loaders: python.langchain.com/docs/concepts/document_loaders
- Text Splitters: python.langchain.com/docs/concepts/text_splitters
- Vector Stores: python.langchain.com/docs/concepts/vectorstores
- Retrievers: python.langchain.com/docs/concepts/retrievers

Evaluation & Quality

- RAGAS — Retrieval-Augmented Generation Assessment: docs.ragas.io
- LangSmith (tracing and evaluation): docs.smith.langchain.com

Embedding Models

- OpenAI Embeddings Guide: platform.openai.com/docs/guides/embeddings
- MTEB Leaderboard (embedding benchmarks): huggingface.co/spaces/mteb/leaderboard



All links verified against current documentation. LangChain APIs evolve quickly — always check the official docs for the latest interface.